

МИНИСТЕРСТВО ЛЕСНОГО ХОЗЯЙСТВА
И ОХРАНЫ ОБЪЕКТОВ ЖИВОТНОГО МИРА
НИЖЕГОРОДСКОЙ ОБЛАСТИ

Государственное бюджетное профессиональное
образовательное учреждение Нижегородской области
«КРАСНОБАКОВСКИЙ ЛЕСНОЙ КОЛЛЕДЖ»

УТВЕРЖДАЮ

Преподаватель

_____ А.В. Торопов

« ____ » _____ 2026 г.

ОТЧЁТ

по лабораторной работе

«Рефакторинг сервисного слоя приложения»

Вариант №10

«Система управления фитнес-клубом»

Выполнила:

Маколикова Анастасия Александровна 24-ИС

Проверил:

преподаватель

Торопов Александр Владимирович

р.п. Красные Баки

2026 г.

ИСХОДНЫЙ КОД РЕФАКТОРИНГА

После выполнения рефакторинга приложение было разделено на отдельные слои.

1. Структура проекта

```
fitness_club/
├── src/
│   ├── models.py
│   ├── exceptions.py
│   ├── repositories.py
│   ├── memory_repositories.py
│   ├── schemas.py
│   ├── service.py
│   └── main.py
├── tests/
│   └── test_service.py
├── requirements.txt
└── fitness.log
```

2. Слой моделей — models.py

Содержит сущности предметной области:

Client

Membership

Workout

Booking

Назначение:

- хранение данных о клиентах;
- хранение данных об абонементх;
- хранение данных о тренировках;
- хранение данных о записях на тренировки.

3. Слой исключений — exceptions.py

Содержит пользовательские исключения:

ClientNotFound

MembershipExpired

MembershipFrozen

WorkoutFull

ValidationError

Назначение:

- обработка ошибок бизнес-логики;
- разделение технических и предметных ошибок.

4. Слой репозитория — repositories.py

Содержит интерфейсы доступа к данным:

ClientRepository

MembershipRepository

WorkoutRepository

BookingRepository

Назначение:

- определение контрактов работы с данными;
- снижение связанности компонентов.

5. Слой хранения данных — `memory_repositories.py`

Содержит реализации репозитория:

`InMemoryClientRepository`

`InMemoryMembershipRepository`

`InMemoryWorkoutRepository`

`InMemoryBookingRepository`

Назначение:

- хранение данных в оперативной памяти;
- реализация интерфейсов репозитория.

6. Слой валидации — `schemas.py`

Содержит схемы Pydantic:

`ClientCreate`

`WorkoutCreate`

Назначение:

- проверка корректности входных данных;
- предотвращение ошибок пользователя.

7. Сервисный слой — `service.py`

Содержит основной класс:

`FitnessClubService`

Реализует:

- регистрацию клиентов;
- оформление абонементов;
- заморозку абонементов;
- создание тренировок;
- запись на тренировки;
- расчёт скидок;
- логирование действий.

8. Точка входа — `main.py`

Назначение:

- создание объектов репозитория;
- внедрение зависимостей;
- демонстрация работы системы.

9. Модуль тестирования — `tests/test_service.py`

Содержит тесты:

`test_booking_success`
`test_client_not_found`
`test_membership_frozen`
`test_workout_full`
`test_discount`

Назначение:

- проверка бизнес-логики;
- контроль корректности работы приложения;
- обеспечение покрытия кода тестами.

ДИАГРАММА КЛАССОВ

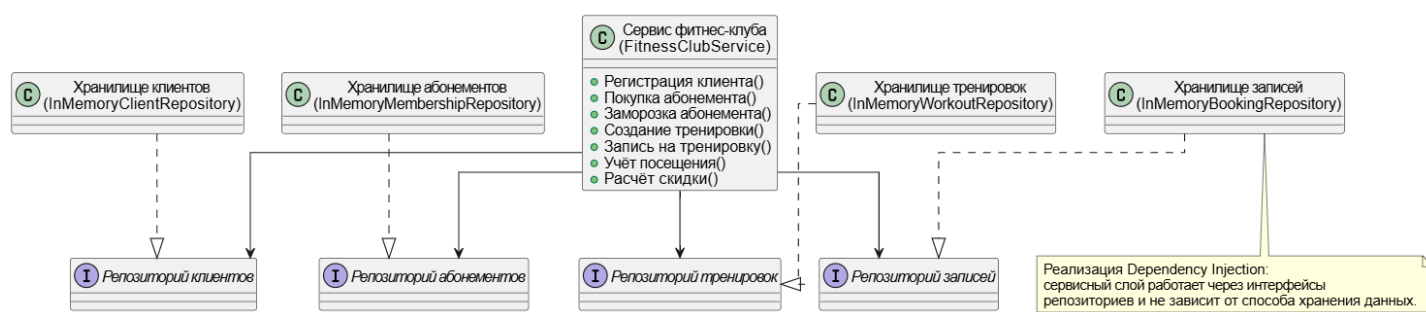


Рисунок 1 – Диаграмма зависимостей системы управления фитнес-клубом

СКРИНШОТЫ ВЫПОЛНЕНИЯ ТЕСТОВ

```

C:\Windows\System32\cmd.exe

C:\Users\Awy\Desktop\practice_PM02_makolikova\Day 14\fitness_club>py -3.12 -m pytest -v
===== test session starts =====
platform win32 -- Python 3.12.10, pytest-9.1.0, pluggy-1.6.0 -- C:\Users\Awy\AppData\Local\Programs\Python\Python312\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\Awy\Desktop\practice_PM02_makolikova\Day 14\fitness_club
plugins: hypothesis-6.155.3, cov-7.1.0, xdist-3.8.0
collected 5 items

tests/test_service.py::test_booking_success PASSED [ 20%]
tests/test_service.py::test_client_not_found PASSED [ 40%]
tests/test_service.py::test_membership_frozen PASSED [ 60%]
tests/test_service.py::test_workout_full PASSED [ 80%]
tests/test_service.py::test_discount PASSED [100%]

===== 5 passed in 0.27s =====

C:\Users\Awy\Desktop\practice_PM02_makolikova\Day 14\fitness_club>

```

Рисунок 2 – Результат выполнения модульных тестов

```

C:\Windows\System32\cmd.exe

C:\Users\Awy\Desktop\practice_PM02_makolikova\Day 14\fitness_club>echo. > src\__init__.py

C:\Users\Awy\Desktop\practice_PM02_makolikova\Day 14\fitness_club>pytest --cov=src --cov-report=term
===== test session starts =====
platform win32 -- Python 3.12.10, pytest-9.1.0, pluggy-1.6.0
rootdir: C:\Users\Awy\Desktop\practice_PM02_makolikova\Day 14\fitness_club
plugins: hypothesis-6.155.3, cov-7.1.0, xdist-3.8.0
collected 5 items

tests/test_service.py ..... [100%]

===== tests coverage =====
coverage: platform win32, python 3.12.10-final-0

Name                               Stmts  Miss  Cover
-----
src\__init__.py                      0      0  100%
src\exceptions.py                   12      0  100%
src\main.py                         20     20   0%
src\memory_repositories.py          29      0  100%
src\models.py                       25      0  100%
src\repositories.py                 29      8   72%
src\schemas.py                     9      9   0%
src\service.py                     56      5   91%
TOTAL                             180     42   77%

===== 5 passed in 0.40s =====

C:\Users\Awy\Desktop\practice_PM02_makolikova\Day 14\fitness_club>

```

Рисунок 3 – Покрытие кода модульными тестами

DIFF СТАРОГО И НОВОГО КОДА

Было

```
class FitnessService:  
  
    def __init__(self):  
        self.clients = []  
        self.memberships = []  
        self.workouts = []
```

Проблемы

- нарушение SRP;
- данные хранятся внутри сервиса;
- нет DI;
- нет репозиториев;
- нет тестируемости.

Стало

```
class FitnessClubService:

    def __init__(
        self,
        client_repo,
        membership_repo,
        workout_repo,
        booking_repo):

        self.client_repo = client_repo
        self.membership_repo = membership_repo
        self.workout_repo = workout_repo
        self.booking_repo = booking_repo
```

Результат

- внедрение зависимостей (Dependency Injection);
- слабая связанность;
- сервис легко тестировать.

Было

```
if client is None:
    return "Клиент не найден"
```

Стало

```
if not client:
    raise ClientNotFound()
```

Результат

Используются собственные доменные исключения.

Было

```
booking_repo.add(booking)
```

Стало

```
if membership.frozen:
    raise MembershipFrozen()

if participants >= workout.max_participants:
    raise WorkoutFull()
```

```
booking_repo.add(booking)
```

Результат

Добавлена бизнес-логика фитнес-клуба.

Было

```
# логирование отсутствует
```

Стало

```
logging.info(
    f"Клиент зарегистрирован: {name}"
)
```

Результат

Добавлен аудит действий пользователей.

АНАЛИЗ: КАКИЕ ПРОБЛЕМЫ БЫЛИ РЕШЕНЫ И КАК ИЗМЕНИЛАСЬ АРХИТЕКТУРА

В процессе рефакторинга были выявлены и устранены основные архитектурные недостатки исходного решения.

Во-первых, была устранена жёсткая связанность между компонентами системы. Вместо хранения данных непосредственно внутри сервисного класса были реализованы репозитории и внедрение зависимостей (Dependency Injection).

Во-вторых, бизнес-логика была отделена от логики хранения данных. Для доступа к данным используются отдельные репозитории, а сервисный слой отвечает только за выполнение бизнес-операций.

В-третьих, была реализована система пользовательских исключений, позволяющая корректно обрабатывать ошибки предметной области: отсутствие клиента, истечение срока действия абонеента, заморозка абонеента и переполнение группы.

Также была добавлена валидация входных данных с использованием библиотеки Pydantic, что повысило надёжность системы.

Для контроля работы приложения внедрено логирование основных действий пользователей. Это позволяет отслеживать регистрацию клиентов, оформление абонеентов и запись на тренировки.

Кроме того, были разработаны модульные тесты с использованием pytest, что позволило проверить корректность работы бизнес-логики и повысить качество программного обеспечения.

До рефакторинга система представляла собой монолитный класс, в котором одновременно выполнялись функции хранения данных и реализации бизнес-логики. Такой подход нарушал принципы SOLID и затруднял сопровождение приложения.

После рефакторинга была реализована многослойная архитектура, состоящая из следующих компонентов:

- слой моделей (models.py);
- слой репозитория (repositories.py);
- реализации репозитория (memory_repositories.py);
- сервисный слой (service.py);
- слой валидации (schemas.py);
- модуль тестирования (tests).

Взаимодействие между слоями осуществляется через интерфейсы репозитория и механизм Dependency Injection. Благодаря этому сервисный слой не зависит от конкретной реализации хранения данных и может работать как с памятью, так и с базой данных без изменения бизнес-логики.